P1810R0
A Quick Look at What P1754 Will Change

Draft Proposal, 2012-07-12

This version:
https://wg21.link/p1810

Author:
Christopher Di Bella

Audience:
EWG, LEWG, LWG

Project:
ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

[P1754] has a single, abundantly clear goal outlined in its name: to change the naming convention for all standard concepts from `PascalCase` to `snake_case`. Examples in P1754 are sadly lacking: it would have been nice to see an algorithm or two with the differences displayed side-by-side. I was curious about what it would look like, so I decided to contrast the two using the library that's benefited from concepts the most: our algorithms library.

Table of Contents

§ 1. Examples

```
// Status quo
template<InputIterator I, Sentinel<I> S, class T, class Proj = identity>
requires IndirectRelation<ranges::equal_to, projected<I, Proj>, T*>
constexpr I find(I first, S last, const T& value, Proj proj = {});

// P1754's suggestion
template<input_iterator I, sentinel_for<I> S, class T, class Proj = identit
requires indirect_relation<ranges::equal_to, projected<I, Proj>, T*>
constexpr I find(I first, S last, const T& value, Proj proj = {});
```

```
// Status quo
template<ForwardIterator I, Sentinel<I> S, class T, class Proj = identity,
         IndirectStrictWeakOrder<const T*, projected<I, Proj>> Comp = range
constexpr I upper_bound(I first, S last, const T& value, Comp comp = {}, Pr

// P1754's suggestion
template<forward_iterator I, sentinel_for<I> S, class T, class Proj = ident
         indirect_strict_weak_order<const T*, projected<I, Proj>> Comp = ra
constexpr I upper_bound(I first, S last, const T& value, Comp comp = {}, Pr
```

```
// Status quo
template<InputIterator I1, Sentinel<I1> S1, RandomAccessIterator I2, Sentir
         class Comp = ranges::less, class Proj1 = identity, class Proj2 = i
requires IndirectlyCopyable<I1, I2> && Sortable<I2, Comp, Proj2> &&
         IndirectStrictWeakOrder<Comp, projected<I1, Proj1>, projected<I2,
constexpr I2 partial_sort_copy(I1 first, S1 last, I2 result_first, S2 resul
                               Comp comp = {}, Proj1 proj1 = {}, Proj2 proj

// P1754's suggestion
template<input_iterator I1, sentinel_for<I1> S1, random_access_iterator I2,
         class Comp = ranges::less, class Proj1 = identity, class Proj2 = i
requires indirect_copyable<I1, I2> && sortable<I2, Comp, Proj2> &&
         indirect_strict_weak_order<Comp, projected<I1, Proj1>, projected<I
constexpr I2 partial_sort_copy(I1 first, S1 last, I2 result_first, S2 resul
                               Comp comp = {}, Proj1 proj1 = {}, Proj2 proj
```

The underscores serve as spaces in the names, which in my opinion, make the concepts significantly easier to read -- particularly in the case of `partial_sort_copy`, which is quite a mouthful. I've also applied the `snake_case` concepts to my numeric algorithms proposal, and it's helped to improve that document's readability there too. I imagine production code using the `snake_case` concepts will benefit even more than the spec.

§ 1.1. Concerns and the status quo

I've seen many people express distaste for the proposed change side due to the fact that 'we've always named our concepts using `PascalCase`'. We haven't: we have a set of tables in the standard library that outline requirements, and we name these requirements `EqualityComparable`, and so on. While they do use `PascalCase`, those names are used to alias tables that appear only to aid in understanding the standard prose, and don't represent any standard identifier.

Secondly, it's easy to come to the conclusion that because there's a named requirement called `EqualityComparable` and a concept called `EqualityComparable`, of course the concept is just a way for us to express our requirements in a compiler-checkable way. This is misguided: the concept `EqualityComparable` goes way deeper than the named requirement `EqualityComparable`. It is left as an exercise to the reader to distinguish the differences between the two. (One should note that the named requirement has been rebranded as *Cpp17EqualityComparable* to signify that it's different to the concept, and that the font face has changed from `verbatim` to *italics*, which means that there shouldn't be any more names in the standard spelt using `PascalCase`.)

Finally, our current templates look like this:

```
template<class InputIterator>
template<typename InputIterator>
template<auto N>
template<int N>
```

They do not look like this:

```
template<Class I>    // error: template type parameter declared using 'clas
template<Typename I> // error: template type parameter declared using 'clas
template<Auto N>     // error: template non-type parameter declared using '
template<Int N>      // error: template non-type parameter declared using '
```

In other words, my take on this, is that P1754 aims to *restore* what we've always done, which is to use a synonym of `typename` to declare a type parameter in `snake_case`, and we name our type parameter in `PascalCase`. When I pointed this out to Herb, he mentioned that this is already articulated in §1.1.3.

§ 1.2. Who's co-authoring it?

All of the major authors for both concepts and ranges have co-authored P1754, and are unanimous that this should happen. Being the architects for these features, it is likely that they collectively and individually have the most experience of anyone with these names. We should solemnly acknowledge this experience, and at least *heavily* experiment with their proposal before digging our heels in and claiming that the aberration in the standard library should remain status quo.

§ 2. Thoughts on the new concept names

P1754 also seeks to alter the spelling of a few concepts, as seen below.

Current	With P1754
`Same`	`same_as`
`CommonReference`	`has_common_reference`
`Common`	`has_common_type`
`Assignable`	`assignable_from`

`Boolean`	`boolean_type`
`StrictTotallyOrdered(With)`	`totally_ordered(_with)`
`StrictWeakOrder`	`weak_order`
`Iterator`	`iterator_type`
`(Sized)Sentinel`	`(sized_)sentinel_for`
`IndirectlyMovable(Storable)`	`indirect_movable(_storable)`
`IndirectlyCopyable(Storable)`	`indirect_copyable(_storable)`
`IndirectlySwappable`	`indirect_swappable`
`IndirectlyComparable`	`indirect_comparable`
`Range`	`range_type`
`View`	`view_type`

§ 2.1. Favoured renamings

Reimagining `Sentinel` as `sentinel_for` makes a lot of sense. I also welcome renaming `Assignable` to `assignable_from`, but wonders if `Constructible`'s name should become `constructible_from` for similar reasons.

It's not clear if choosing to rename `IndirectlyCopyable` to `indirect_copyable` was intentional, so I'd like to provide motivation *for* this case, in the event it wasn't. While `indirectly_copyable` reads nicer than `indirect_copyable`, and we could argue that the `indirect_` concepts belong to [indirectcallable] and the `indirectly_` concepts live in [alg.req], I think this is just happenstance. For example, when describing the requirements for the C++17 numeric algorithms, I needed to define a concept called `commutative_invocable`, which has an indirect callable counterpart that would likely live in [indirectcallable]. It's very tempting to name this `indirectly_commutative` instead of `indirect_commutative_invocable`, which immediately blurs the line between what's `indirect_` and what's `indirectly_`. In light of that, to avoid confusion, I encourage exactly one of `indirect_` or `indirectly_`.

§ 2.2. Bikeshedding

There are a few names that I'm not so keen on, and would like to see change, as shown below.

P1754's name	My alternative	Comment
`has_common_reference`	TBD	`has_plus` is the epitome of our fears when designing concepts (see the design ideals of [N3351]). As
`has_common_type`	`common_with` or `in_common_with` or	

	`common`	such, we should aim to avoid having `has_anything` in the IS*, as it sets a dangerous precedent: if `has_common_type` exists, then the case against `has_plus` becomes a lot harder. Since `Common` refines `CommonReference`, the name `has_common_type` doesn't really fit: it's a clunky, syntactic name that that implies `common_type_t<T, U>` is well-formed (and [meta.trans.other-Note-A] doesn't concern itself with `common_reference`). *At the very least, tangible names shouldn't be prefixed with `has_` or `is_`, etc., for this reason.
`boolean_type`	`boolean` or `context_bool` or `contextually_bool` or `context_boolean` or `contextually_boolean` or `as_if_bool`	Although the phrase '`T` models `boolean_type`' reads okay, it clashes with existing standard type names, such as `common_type`, `size_type`, `difference_type`, `value_type`, and `element_type`. Despite there being only a few concepts that end with `_type`, and despite most of them not really being necessary outside of defining other concepts, it is possible that these concepts are mistaken for types. For example, a user might conclude that `boolean_type` is the standard spelling for a user-defined boolean type to avoid clashing with any non-standard boolean types. After all, we've already set a precedent here.

		Corollary: `boolean_concept` and `boolean_c` are 'entity encoding' in identifiers; something we try to avoid, and so I discourage this usage too. Of the suggested alternatives, I prefer `contextually_bool`.

`iterator_type`
 	
`range_type`
	`ranges::iterator`
	
	`ranges::range`
		The `_type` needs to go, for similar reasons as `boolean_type`.
		
		As P1754 points out, `std::iterator` is already taken by a (deprecated) type. Unless we're willing to remove the current `std::iterator` entity and introduce a new `std::iterator` entity *in the same standard*, we can't choose `std::iterator`.
		
		We can, however, choose `std::ranges::iterator`, which is slightly more compelling, as it has similar motivation for being in this namespace as our new algorithms (and we can hopefully eject these into namespace `std` one day). This implies that all P0896-introduced concepts will also need to be moved across as well.

`view_type`
 	
`viewable_range`
	`viewable_range`
	
	`view_adaptable`
		All types that model `view_type` also model `range`, so I find the name `viewable_range` to be misleading, since `viewable_range` *potentially* refines `view_type`. (Note that it's not clear to me how refinement

works when disjunctions are involved.)

From [range.refinements], we see that "the `ViewableRange` concept specifies the requirements of a `Range` type that can be converted to a `View` safely". Actual usage of `ViewableRange` shows that it's used for adapting ranges into views. I suggest that the name of `viewable_range` concept reflect this canonical use-case, and is of the opinion that `view_adaptable` is more appropriate for this reason.

This then frees up the name `viewable_range`, which can now be used as the new spelling for `View` (since `view_type` has the same issues as `boolean_type`, etc.). Furthermore, it will help cement the idea that all views are ranges.

§ 3. Conclusion

Some of the names, such as `has_common_reference` and `range_type` are a bit controversial, and should be reviewed, but I want to make my position unambiguously clear: I do not think that the current P1754 names should be a showstopper for gaining consensus.

I understand that there's concern around renaming these concepts -- and P1754's authors are also aware of this -- but I fear that the concern is misplaced. We should embrace the change that P1754 seeks to make, so that our code doesn't become too squashed, like `partial_sort_copy`.

At the very least, please write a non-trivial example with the new names, so that you've got something to back you up when you say "I don't like it".

§ References

§ Normative References

[ALG.REQ]

Common algorithm requirements. URL: http://eel.is/c++draft/alg.req

[INDIRECTCALLABLE]

Indirect callable requirements. URL: http://eel.is/c++draft/indirectcallable

[meta.trans.other-Note-A]

[meta.trans.other] Note A. URL: http://eel.is/c++draft/meta.trans.other#3

[RANGE.REFINEMENTS]

Other range refinements. URL: http://eel.is/c++draft/range.refinements#4

§ Informative References

[N3351]

Bjarne Stroustrup; Andrew Sutton. A Concept Design for the STL. URL: https://wg21.link/n3351

[P1754]

Herb Sutter; et al. Rename concepts to standard_case for C++20, while we still can. URL: https://wg21.link/p1754